



Runnable



Runnable` representa **una unidad de trabajo**: el bloque de código que un hilo ejecutará de forma concurrente.

Explicación

Runnable es una **interfaz funcional**, lo que significa que solo declara un método abstracto:

```
void run();
```

Por eso se puede usar con **expresiones lambda**, lo que hace su utilización mucho más cómoda.

```
public class DemoRunnableLambda {  
    public static void main(String[] args) {  
        Runnable tarea = () → {  
            System.out.println("Hola desde: " + Thread.currentThread().getName  
());  
        };  
  
        Thread hilo = new Thread(tarea);  
        hilo.start();  
    }  
}
```

¿Qué significa unidad de trabajo?

Cuando hablamos de Runnable solemos denominarlo “unidad de trabajo” o similares.

Esto es porque con esta interfaz vamos a definir el código con el que el hilo va a trabajar, y nada más.

`Runnable` **define la tarea,**

`Thread` **la ejecuta.**

Un objeto que implementa Runnable **NO ES un hilo.**

Es solo un “contenedor de trabajo”.

Para obtener concurrencia real, debemos entregarle esa tarea a un `Thread` y llamarle a **start()**:

```
Thread t = new Thread(new MiTarea());  
t.start(); // aquí nace el hilo real
```

El código quedará definido dentro del método `.run()`.

Muy importante tener claro que un Runnable define una tarea, pero solo se ejecuta concurrentemente cuando lo recibe un objeto `Thread` y se invoca `start()` sobre él.

Llamar a `run()` nunca crea un nuevo hilo.

El método `run()`

El método:

- no devuelve nada,
- no recibe parámetros,
- contiene el código que ejecutará el hilo cuando sea lanzado.

Pero atención:

Llamar a `run()` directamente NO crea concurrencia.

Es simplemente una llamada normal a un método en el hilo actual.

Ejemplo ilustrativo:

```

class MiTarea implements Runnable {
    @Override
    public void run() {
        System.out.println("Ejecutando en: " + Thread.currentThread().getName());
    }
}

public class DemoRunVsStart {
    public static void main(String[] args) {
        Runnable tarea = new MiTarea();
        Thread hilo = new Thread(tarea, "Hilo-Real");

        System.out.println("Hilo actual: " + Thread.currentThread().getName());

        System.out.println("\nLlamando a run():");
        tarea.run(); // o hilo.run();
        // → NO hay concurrencia: se ejecuta en el hilo main

        System.out.println("\nLlamando a start():");
        hilo.start(); // → concurrencia real
    }
}

```

Ventaja frente a heredar de Thread

Herencia directa:

```

class MiHilo extends Thread {
    public void run() {}
}

```

funciona, pero **no es la opción preferible**, porque:

- Java no tiene herencia múltiple → si ya heredas de `Thread`, no puedes heredar de nada más.

- Runnable separa la **lógica de la tarea** del **mecanismo de ejecución**, lo que encaja mejor con pools, ejecutores, etc.

Por eso:

| Runnable es la forma estándar y flexible de definir tareas concurrentes en Java.

Runnable con Thread (forma recomendada)

```
class MiTarea implements Runnable {
    @Override
    public void run() {
        System.out.println("Ejecutando tarea en: " + Thread.currentThread().get
Name());
    }
}

public class DemoRunnableBasico {
    public static void main(String[] args) {
        Runnable tarea = new MiTarea();
        Thread hilo = new Thread(tarea);

        hilo.start(); // concurrencia real
    }
}
```

Runnable con lambda

Dado que Runnable es una interfaz funcional, podemos escribir:

```
Thread t = new Thread(() → {
    System.out.println("Desde lambda: " + Thread.currentThread().getName());
});

t.start();
```